

问题四思路

这个脚本的核心思路是：

把第四问转化为“候选任务窗口选择问题”，
先生成所有可能执行的射击/拍照任务，
再用0-1整数规划从中选出最优组合。

它的整体流程可以概括为：

```
graph TD; A[读取问题3的10Hz融合轨迹] --> B[↓]; B --> C[用卡尔曼模型估计速度、加速度]; C --> D[↓]; D --> E[读取附件4中的射击目标和拍照目标]; E --> F[↓]; F --> G[逐目标、逐时刻判断是否满足作业条件]; G --> H[↓]; H --> I[生成射击/拍照候选任务窗口]; I --> J[↓]; J --> K[建立0-1整数规划模型]; K --> L[↓]; L --> M[加入时间冲突、射击次数、拍照角度约束]; M --> N[↓]; N --> O[最大化期望任务数]; O --> P[↓]; P --> Q[输出任务安排表和 result.xlsx]
```

1. 读取输入数据

脚本首先读取两类数据：

```
TRAJECTORY_PATH = outputs/问题三/problem3_trajectory_10hz_kalman.csv  
TARGET_PATH = 数据以及可视化/附件4.xlsx
```

其中，轨迹文件来自问题3的卡尔曼融合结果，至少需要包含：

```
time_s  
x_kalman_m
```

```
y_kalman_m
```

附件4中包含射击目标和拍照目标的位置。脚本用 `load_trajectory()` 读取轨迹，用 `load_targets()` 读取目标点。

2. 估计机器人运动状态

问题3轨迹主要给出了机器人位置，但第四问需要判断速度、加速度约束。

因此脚本使用 `build_motion_table()` 进一步计算：

```
x, y  
vx, vy  
ax, ay  
speed  
accel
```

它不是简单差分，而是调用 `kalman_motion_state()`，使用一个六维状态：

```
[x, vx, ax, y, vy, ay]
```

也就是用匀加速度卡尔曼模型估计运动状态。这样做的好处是：避免直接差分导致速度、加速度被噪声放大。

3. 生成射击候选任务

射击任务的约束包括：

```
距离在 5m 到 30m 之间；  
速度不超过 2.0m/s；  
加速度不超过 1.5m/s2；  
射击前需要连续 1.5s 满足条件。
```

代码中对应参数是：

```
SHOOT_DISTANCE_RANGE_M = (5.0, 30.0)  
SHOOT_SPEED_MAX_MPS = 2.0
```

```
ACCEL_MAX_MPS2 = 1.5
SHOOT_PREP_POINTS = 15
```

因为轨迹是10Hz，所以15个点对应：

$$15 \times 0.1s = 1.5s$$

`build_shoot_candidates()` 会对每个射击目标、每个时间点判断是否满足约束。如果某个执行时刻之前连续15个点都满足条件，就生成一个射击候选任务窗口：

```
[start_idx, exec_idx]
```

也就是：

开始准备时刻 → 射击执行时刻

每个候选任务保存目标编号、任务类型、开始时间、执行时间、执行距离、速度、加速度和任务权重。

4. 生成拍照候选任务

拍照任务的约束包括：

距离在 10m 到 40m 之间；
速度不超过 1.5m/s；
加速度不超过 1.5m/s²；
拍照前需要连续 0.5s 满足条件；
同一目标多次拍照角度差至少 60°。

代码参数是：

```
PHOTO_DISTANCE_RANGE_M = (10.0, 40.0)
PHOTO_SPEED_MAX_MPS = 1.5
ACCEL_MAX_MPS2 = 1.5
PHOTO_PREP_POINTS = 5
PHOTO_MIN_ANGLE_DIFF_DEG = 60.0
```

因为10Hz下5个点对应：

$$5 \times 0.1s = 0.5s$$

`build_photo_candidates()` 的逻辑和射击类似，只是多计算了一个拍照方向角：

```
angle = atan2(ty - y, tx - x)
```

也就是机器人看向目标的方向角。每个拍照候选任务都会记录这个角度，后面用于判断同一目标的多次拍照是否满足60°角度差。

5. 建立0-1整数规划模型

每一个候选任务都对应一个二元变量：

```
x_i = 1 表示选择该候选任务  
x_i = 0 表示不选择该候选任务
```

脚本用 `solve_selection()` 建立并求解整数规划。

目标函数是最大化期望任务数。代码中设置：

```
SHOOT_WEIGHT = 0.85  
PHOTO_WEIGHT = 1.0
```

所以目标函数相当于：

```
max 0.85 × 射击任务数 + 1.0 × 拍照任务数
```

由于 `scipy.optimize.milp()` 默认是最小化，所以代码中使用负权重：

```
c = -weights
```

从而实现最大化。

6. 加入三类核心约束

约束1：时间片互斥

机器人同一时刻只能执行一个任务。

脚本对每个10Hz时间点建立约束：

所有覆盖该时间点的候选任务，最多只能选一个

即：

$$\sum x_i \leq 1$$

这避免了两个任务的准备/执行时间重叠。

约束2：每个射击目标最多射击一次

对于同一个射击目标，所有候选射击窗口最多只能选择一个：

$$\sum x_i \leq 1$$

这表示一个射击目标完成一次即可，不重复射击。

约束3：同一拍照目标角度差不少于60°

对同一个拍照目标，脚本两两比较候选拍照任务的方向角。

如果两个候选任务角度差小于60°，则添加互斥约束：

$$x_i + x_j \leq 1$$

角度差使用的是圆周角差：

$$\text{abs}((\text{left} - \text{right} + 180) \% 360 - 180)$$

这样可以正确处理 350° 和 10° 这种跨越0°/360°的情况。

7. 求解并输出结果

整数规划求解完成后，脚本把被选中的任务按照执行时间排序，然后生成结果表。

输出内容包括：

序号
目标编号
任务类型
开始准备时刻
任务执行时刻
执行距离
执行速度
执行加速度
拍照方向角
目标函数权重

然后保存为：

outputs/问题四/problem4_selected_tasks.csv
outputs/问题四/problem4_summary.json
outputs/问题四/result_problem4.xlsx

同时，它还会尝试把结果写回 `result.xlsx`，但只写入前五列：

序号
目标编号
任务
开始准备时刻
任务执行时刻

这样可以避免破坏模板中其他区域，特别是红色说明文字。

一句话总结

这个脚本的思路是：

先根据距离、速度、加速度和准备时间生成所有可行的射击/拍照候选任务，再用0-1整数规划在“时间不冲突、射击不重复、拍照角度不同”的约束下，选择使期望任务数最大的任务组合。

它是一个比较完整的第四问“任务优化调度”求解框架。